SYN Flood Virtual Lab — Build Plan & Prompts

Oct 12, 2026 · @Mario_Bunny

1. What we're building

In one day we build a browser-only **SYN Flood Virtual Lab**: plain HTML, CSS and JavaScript, no backend and no real network traffic. A student walks through it step by step and watches a simulated server lose availability as its connection queue fills, then recover when defences are switched on. The final deliverable is exactly three files, `index.html`, `style.css` and `script.js`, as the brief requires.

Student journey (from the brief): Before You Begin → Theory → Baseline traffic → SYN Flood simulation → Observe impact → Stop → Mitigation → Comparison → Quiz → Completion popup.

How five people work at once without merge pain:

- **Contract first.** Section 5 fixes every function name, event, HTML `id`, CSS class and data shape before anyone starts. Nobody renames anything in it without telling Manvith.
- **One owner per file.** Each person edits only their own files, so Git never has conflicts.
- **Mock engine.** The visual developers build against a fake engine (`?mock=1` in the URL) until the real one is ready.
- **Build step.** A small script concatenates the separate CSS and JS files into the final `style.css` and `script.js`.

The brief itself (the PDF from ma'am) goes into the repo as `docs/brief.pdf` so every Claude Code session can read it.

2. Team and ownership

Each person gets about six hours of work and owns separate files.

Member	Role	Owns	Tools	Hours
Manvith Karkera	Lead developer: simulation engine, step workflow, quiz logic, reset, completion popup, final merge	`CONTRACT.md`, `index.html`, `js/engine.js`, `js/workflow.js`, `js/quiz.js`, `js/mock-engine.js`, `build.js`	Claude Code	~6
Pranav Koradiya	Developer: network topology and packet animation, handshake demo, live dashboard, chart, log, comparison table	`js/visuals.js`, `js/dashboard.js`	Claude Code	~6
Yash Masaye	Designer: visual identity, page layout, header, buttons, forms, stepper, guidelines and theory screens, modals, responsive and accessibility pass	Figma mockup, `css/base.css`	Figma + Claude.ai chat	~6
Sanika Lobo	Designer: simulation components (topology nodes, packets, queue grid, metric cards, server badges, log, comparison, quiz), icons, QA testing	Figma components, `css/lab.css`, `js/icons.js`	Figma + Claude.ai chat	~6

Member	Role	Owns	Tools	Hours
Meet Mangaonkar	Content and creative: all lab text, theory, analogy, step instructions, quiz with explanations; README; demo video; presentation slides	`js/content.js`, `README.md`, demo video, slides	Claude.ai chat (no Claude Code needed)	~6

3. Build-day schedule

Times are hours from the start of the build day (0:00 = when you all begin). Codes like **MK1** refer to the prompts in Sections 6–9.

Time	Manvith	Pranav	Yash	Sanika	Meet
0:00–0:30	**MK0** scaffold repo, push contract	Read Section 5, clone repo	Read Sections 1 and 5	Read Sections 1 and 5	Read Sections 1 and 5
0:30–2:00	**MK1** simulation engine	**PK1** topology + packets (mock mode)	**YM1** Figma mockup of whole page	**SL1** component mockups + icons	**MM1** content part 1
2:00–3:30	**MK1** finish + self-test, start **MK2**	**PK1** finish, start **PK2** dashboard	**YM2** `base.css`	**SL2** `lab.css`	**MM1** content part 2 (quiz)
3:30–4:00	**Sync 1 (all):** everyone pushes; Manvith runs the build; look at the page together and agree on fixes				
4:00–6:00	**MK2** workflow, quiz, reset, completion	**PK2** dashboard, chart, log, comparison	**YM2** finish, **YM3** responsive + accessibility	**SL2** polish on the real page	**MM2** README + slides
6:00–7:00	**MK3** integration + build	**PK3** bug-fix pass	Run QA checklist (Section 10), report bugs	Run QA checklist, report bugs	**MM3** record demo takes
7:00–8:00	**Sync 2:** final fixes, freeze `dist/`	Final fixes	Final visual fixes	Final visual fixes	Edit and export demo video

Rule for bug reports: post in the team group as *"where / what happened / what should happen / screenshot"* and tag the file owner.

4. Repo layout and merging

While building, the CSS and JS stay split into one file per owner; `build.js` stitches them into the three final files in `dist/`.

```
syn-flood-lab/
├── docs/brief.pdf        the professor's brief (Manvith adds it)
├── CONTRACT.md           Section 5 of this doc, pasted verbatim (Manvith)
├── index.html            skeleton with every id (Manvith); designers may add
classes, never rename ids
├── css/
|   ├── base.css          Yash: tokens, layout, typography, buttons, forms,
stepper, theory, modals
|   └── lab.css           Sanika: topology, packets, queue, dashboard, log,
comparison, quiz
├── js/                   loaded in THIS order
|   ├── content.js        Meet: all text and quiz data (Lab.Content)
|   ├── icons.js          Sanika: inline SVG icons (Lab.Icons)
|   ├── engine.js         Manvith: simulation model (Lab.Engine)
|   ├── mock-engine.js    Manvith: fake engine for ?mock=1, dev only, not
shipped
|   ├── visuals.js        Pranav: topology, packets, handshake demo
(Lab.Visuals)
|   ├── dashboard.js      Pranav: metrics, chart, log, comparison
(Lab.Dashboard, Lab.Log)
|   ├── quiz.js           Manvith: quiz rendering and scoring (Lab.Quiz)
|   └── workflow.js       Manvith: boots the app, steps, buttons, reset,
completion (loads last)
├── build.js              node build.js → dist/index.html, dist/style.css,
dist/script.js
├── tests/checklist.md
└── dist/                 FINAL DELIVERABLE (3 files)
```

Git workflow

1. Manvith creates the GitHub repo after **MK0** and adds everyone as collaborators.
2. Each person works on their own branch: `manvith`, `pranav`, `yash`, `sanika`, `meet`.
3. At each sync, everyone pushes and Manvith merges all branches into `main`. Because files never overlap, merges are conflict-free.
4. Everyone pulls `main` after each sync so they see the latest whole page.

Running it: open `index.html` directly in Chrome (no server needed). Add `?mock=1` to the URL to use the fake engine. Run `node build.js` to produce `dist/`.

5. Shared contract (CONTRACT.md)

This section is copied verbatim into `CONTRACT.md`. Every prompt tells Claude to read it first. If something here must change, Manvith updates it and announces it in the group.

5.1 JavaScript namespaces

No ES modules and no libraries (the page must open via `file://`). Every JS file uses this wrapper and only adds its own key to `window.Lab`:

```
window.Lab = window.Lab || {};
(function (Lab) {
  'use strict';
  // private helpers here
  Lab.Engine = { /* public API */ };
})(window.Lab);
```

Load order: `content.js`, `icons.js`, `engine.js`, `mock-engine.js` (dev only), `visuals.js`, `dashboard.js`, `quiz.js`, `workflow.js`. Only `workflow.js` runs code on `DOMContentLoaded`; every other file only defines functions.

5.2 Engine API (Lab.Engine, owner: Manvith)

```
const engine = Lab.Engine.create(overrides?);  // overrides merge into
DEFAULTS (5.4)
engine.runBaseline();            // legitimate traffic only, 20 simulated
seconds
engine.runAttack(level);         // 'low' | 'medium' | 'high'; 40 simulated
seconds; uses current mitigation
engine.stop();                   // clears the timer immediately; keeps all
state for study
engine.reset();                  // stops, clears
queue/metrics/history/mitigation, reseeds RNG
engine.setMitigation({ synCookies, rateLimit, shortTimeout, firewall }); //
booleans; only while not running
engine.isRunning();              // boolean
engine.getState();               // deep copy of State (5.3)
engine.getSummary();             // RunSummary of the most recent run, or
null
engine.on(eventName, handler);   // returns an unsubscribe function
```

Event	Payload	Emitted when
`tick`	State	Every simulated second while running
`packet`	`{ id, type, source, outcome }`	A simulated packet worth animating (max 12 per tick, sampled)
`serverStateChange`	`{ from, to, t }`	Server state changes (5.4 thresholds)
`log`	`{ t, level, message }`	Meaningful model changes, e.g. queue passes 50/90/100%
`runEnded`	RunSummary	Run reaches its duration or `stop()` is called

Packet fields: `type` is `'syn' | 'synack' | 'ack' | 'rst'`; `source` is `'legit' | 'attack'`; `outcome` is `'queued' | 'completed' | 'rejected' | 'timeout' | 'filtered' | 'cookie'`. Log `level` is `'info' | 'success' | 'warn' | 'danger'`.

5.3 Data shapes

```
State = {
  t: 0,                          // simulated seconds since run start
  running: false,
  mode: 'idle',                  // 'idle' | 'baseline' | 'attack'
  level: null,                   // 'low' | 'medium' | 'high' | null
  params: { ...DEFAULTS },
  mitigation: { synCookies: false, rateLimit: false, shortTimeout: false,
firewall: false },
  queue: { capacity: 100, legit: 0, attack: 0 },   // occupied half-open
slots by source
  metrics: {
    synIncoming: 0,              // SYNs that arrived this second
    synTotal: 0,
    pending: 0,                  // half-open connections right now
    legitAttempted: 0, legitCompleted: 0,          // cumulative for this run
    rejected: 0, timedOut: 0, filtered: 0,         // cumulative
    queueUtil: 0,                // % 0-100
    cpu: 0,                      // % 0-100
    responseTimeMs: 40,
    availability: 100            // % legit success over the last 5 simulated
seconds
```

```
  },
  serverState: 'Healthy',        // 'Healthy' | 'Under Pressure' | 'Degraded'
| 'Unavailable'
  history: []                    // [{ t, pending, queueUtil, cpu,
responseTimeMs, availability }]
};

RunSummary = {
  mode, level, mitigation,       // what was run
  duration, peakPending, avgQueueUtil, peakCpu, avgResponseTimeMs,
  legitSuccessRate,              // % legitCompleted / legitAttempted for the
whole run
  minAvailability, avgAvailability, finalServerState, stoppedEarly
};
```

5.4 Simulation model

All numbers are educational estimates and must be labelled **Simulated data** in the UI.

Parameter	Default
Tick	500 ms real time = 1 simulated second
Queue capacity	100 half-open slots
Legitimate traffic	5 SYN/s, every one completes its handshake after 1 s (if it got a slot)
Pending timeout	30 s (5 s with *Shorter timeouts*)
Attack Low	4 SYN/s, 50% never complete
Attack Medium	15 SYN/s, 80% never complete
Attack High	50 SYN/s, 95% never complete
Rate limiting	Accept at most 8 SYN/s from the traffic generator; the rest are `filtered`
Firewall filtering	Drops 70% of attack SYNs (`filtered`); wrongly drops 2% of legit SYNs
SYN cookies	When the queue is full, new SYNs get a stateless SYN-ACK (`cookie`) instead of being rejected: no slot used, legit still completes, extra CPU cost

Parameter	Default
Randomness	Seeded PRNG (mulberry32, seed 1337) so every run with the same settings gives the same result

Cause and effect per tick: expire timed-out entries → generate arrivals → apply firewall and rate limit → each remaining SYN takes a free slot, else becomes a cookie (if enabled) or is `rejected` → legit entries complete after 1 s and free their slot → recompute metrics → derive server state.

Formulas (starting point; Manvith may tune constants):

- `queueUtil = pending / capacity × 100`
- `cpu = min(100, 8 + 0.5 × pending + 0.3 × synIncoming + 0.4 × cookiesThisSecond)`
- `responseTimeMs = 40 + 1500 × (queueUtil / 100)³ + 6 × max(0, cpu − 60)`
- `availability` = legit completed ÷ legit attempted over the last 5 s × 100 (100 if no attempts)

Server state (checked top to bottom): Unavailable if availability < 15 · Degraded if availability < 70 or queueUtil ≥ 95 · Under Pressure if availability < 95 or queueUtil ≥ 60 or cpu ≥ 70 · otherwise Healthy.

Target outcomes the engine must reproduce: Baseline → Healthy, ~100% availability. Low → Under Pressure, availability ≥ 90%. Medium → Degraded, availability between 15% and 60%. High → Unavailable, availability < 15%. Medium with any single defence → availability clearly better than Medium without. Medium with all four defences → Healthy or Under Pressure with availability ≥ 90%. No configuration claims 100% protection.

5.5 HTML ids (owner: Manvith's skeleton)

Area	ids
Header	`lab-header`, `btn-theory`, `btn-stop`, `btn-reset`, `btn-restart`
Before You Begin	`screen-guidelines`, `lab-objective`, `guidelines-list`, `ack-checkbox`, `btn-start`, `btn-view-theory`

Area	ids
Theory	`section-theory`, `theory-content`, `handshake-demo`, `btn-play-handshake`
Stepper	`stepper`, `step-list` (8 `li` with `data-step="1..8"` and `data-status`), `step-panel`, `step-title`, `step-instructions`, `step-actions`
Topology	`topology`, `topology-svg`, `component-info`
Controls	`btn-baseline`, `attack-level` (select: low/medium/high), `btn-start-flood`, `btn-stop-sim`
Dashboard	`dashboard`, `server-state`, `notifications` (aria-live), `chart-history` (canvas); 8 cards with `data-metric="syn-incoming
Mitigation	`mitigation`, `mit-syn-cookies`, `mit-rate-limit`, `mit-short-timeout`, `mit-firewall`, `btn-run-mitigated`
Comparison	`comparison`, `compare-table`, `compare-explanation`
Quiz	`quiz`, `quiz-container`, `quiz-score`
Log	`log`, `log-list`
Modals	`modal-complete` (dialog) with `complete-baseline`, `complete-attack`, `complete-mitigated`, `complete-score`, `complete-status`, `btn-review`, `btn-restart-lab`; `modal-confirm` (dialog) with `confirm-message`, `btn-confirm-yes`, `btn-confirm-no`

`data-status` values on steps: `locked | ready | in-progress | completed`.

5.6 CSS tokens and classes

Yash defines every token in `:root` in `base.css`; everyone else only uses them.

```
--color-bg  --color-surface  --color-surface-2  --color-text  --color-muted
--color-border
--color-primary  --color-primary-contrast  --color-focus
--color-legit  --color-attack  --color-pending  --color-rejected  --color-
completed
--color-healthy  --color-pressure  --color-degraded  --color-unavailable
--font-sans  --font-mono  --radius  --radius-sm  --shadow
--space-1 --space-2 --space-3 --space-4 --space-5 --space-6
```

Owner	Classes
Yash (`base.css`)	`.container` , `.section` , `.card` , `.btn` , `.btn--primary` , `.btn--danger` , `.btn--ghost` , `.field` , `.checkbox` , `.toggle` , `.stepper` , `.step` , `.step--locked/--ready/--in-progress/--completed` , `.theory` , `.guidelines` , `.modal` , `.toast` , `.sim-badge` , `.visually-hidden`
Sanika (`lab.css`)	`.topology` , `.node` , `.node--client/--generator/--server/--queue/--status` , `.node.is-selected` , `.link` , `.packet` , `.packet--legit/--attack/--synack/--ack/--rejected/--filtered/--cookie` , `.queue-grid` , `.queue-slot` , `.queue-slot--free/--legit/--attack` , `.metric` , `.metric__label` , `.metric__value` , `.progress` , `.progress__bar` , `.server-badge` , `.server-badge--healthy/--pressure/--degraded/--unavailable` , `.chart` , `.log-entry` , `.log-entry--info/--success/--warn/--danger` , `.compare-table` , `.quiz-question` , `.quiz-option` , `.quiz-option.is-correct` , `.quiz-option.is-wrong` , `.quiz-feedback` , `.component-info` , `.mitigation-item`

5.7 Content and icon shapes

```
Lab.Content = {
  title, objective, disclaimer,           // strings
  guidelines: [ /* the 9 strings from the brief */ ],
  theory: { tcp, handshake: [{ name: 'SYN', text }, { name: 'SYN-ACK', text
}, { name: 'ACK', text }],
            synFlood, availability, analogy },
  components: { client, generator, server, queue, status },   // each {
title, text }
  steps: [ { id: 1, title, instructions } /* ...8 steps */ ],
  serverStates: { Healthy, 'Under Pressure', Degraded, Unavailable },  //
one-line explanations
  mitigations: { synCookies, rateLimit, shortTimeout, firewall },     // each
{ title, text, limitation }
  quiz: [ { id, question, options: [4 strings], answer: 0, explanation } /* 6
questions */ ],
  completion: { title, message }
};
Lab.Icons = { client, generator, server, queue, status, shield };  // each an
inline SVG string, 24x24 viewBox, stroke="currentColor"
```

5.8 Frontend module APIs

`workflow.js` is the only file that wires buttons; the modules below only render.

```
// Pranav: visuals.js
Lab.Visuals.init(engine);            // draw topology into #topology-svg,
subscribe to tick/packet events
Lab.Visuals.playHandshakeDemo();     // animate SYN → SYN-ACK → ACK inside
#handshake-demo
Lab.Visuals.reset();                 // remove packets, empty queue grid,
clear selection

// Pranav: dashboard.js
Lab.Dashboard.init(engine);          // subscribe to
tick/serverStateChange/log; render cards, badge, chart
Lab.Dashboard.notify(message, level);// toast + #notifications aria-live text
Lab.Dashboard.renderComparison(runs);// runs = { baseline, attack, mitigated
} RunSummaries (any may be null)
Lab.Dashboard.reset();
Lab.Log.add(message, level);         // timestamped entry in #log-list; level
as in 5.2
Lab.Log.clear();

// Manvith: quiz.js
Lab.Quiz.init(containerEl, questions, onFinished);   // onFinished({ score,
total })
Lab.Quiz.reset();
```

6. Manvith's prompts (Claude Code)

Before **MK0**: create an empty folder `syn-flood-lab`, put the brief in `docs/brief.pdf`, and paste all of Section 5 into `CONTRACT.md`. Run Claude Code inside that folder.

MK0 · Scaffold (0:00–0:30)

```
We are building "SYN Flood Virtual Lab", a browser-only educational simulation: HTML, CSS and vanilla JS, no backend, no libraries, no real network traffic. Five people build separate files in parallel; at the end build.js concatenates them into exactly three files: dist/index.html, dist/style.css, dist/script.js.

Read docs/brief.pdf and CONTRACT.md completely before writing anything. CONTRACT.md is the source of truth for every id, class, function name, event and data shape. Do not invent alternatives.

Your task is ONLY the scaffold that lets the team start. Do not implement feature logic.

1. index.html: a complete semantic skeleton of the whole lab in journey order: header → "Before You Begin" screen → Theory → stepper + step panel → topology → simulation controls → dashboard → mitigation → comparison → quiz → experiment log → the two <dialog> modals. Include EVERY id from CONTRACT §5.5 exactly once, the 8 step <li> elements with data-step and data-status="locked", the 8 dashboard cards with data-metric, the 4 mitigation checkboxes with <label>s, a select#attack-level with low/medium/high, and a visible "Simulated data" .sim-badge on the dashboard. Use proper headings, <main>, <section aria-labelledby>, and button type="button". Put an HTML comment above each section naming its owner. In <head> link css/base.css then css/lab.css. Before </body> load the JS files with plain <script src> tags in the CONTRACT §5.1 order (no type="module"; it must work from file://).
2. Stub files for every file in the repo tree: css/base.css, css/lab.css, js/content.js, js/icons.js, js/engine.js, js/visuals.js, js/dashboard.js, js/quiz.js, js/workflow.js. Each JS stub uses the §5.1 wrapper, has a header comment with owner + purpose, and exposes the full public API from §5.2/§5.7/§5.8 as no-op functions that never throw (Lab.Engine.create must return an object with every method; Lab.Content and Lab.Icons must have every key with placeholder values).
3. js/mock-engine.js: a fake engine implementing the full §5.2 API and emitting tick, packet, serverStateChange, log and runEnded events on a 500 ms timer with plausible numbers (baseline stays Healthy; attack fills the queue over ~10 ticks and walks the server state down to Degraded/Unavailable depending on level; any mitigation enabled makes it recover). It replaces Lab.Engine.create ONLY when location.search contains mock=1. Comment it clearly as dev-only.
4. build.js (Node, no dependencies, works on Windows/macOS/Linux): creates dist/; writes dist/style.css = base.css + lab.css; writes dist/script.js = the JS files in contract order EXCLUDING mock-engine.js, each preceded by a banner comment; writes dist/index.html from index.html with the two CSS links
```

```
replaced by one <link rel="stylesheet" href="style.css"> and all script tags
replaced by one <script src="script.js"></script>.
5. README.md (how to open, ?mock=1, node build.js, file ownership table),
.gitignore, tests/checklist.md (empty heading).

Then run node build.js and verify, with a short Node script that loads
dist/script.js in a vm using a stubbed window and document (no npm packages),
that no stub throws on load. Finally print the git commands to init the repo,
commit, and create branches manvith, pranav, yash, sanika, meet.
```

MK1 · Simulation engine (0:30–2:30)

```
Read CONTRACT.md and docs/brief.pdf. Implement js/engine.js: the real
Lab.Engine described in CONTRACT §5.2–5.4. This is the most important file in
the project: the server must become unavailable BECAUSE the simulated queue
fills, never because code sets a label.

Requirements:
- Exact public API, events and State/RunSummary shapes from the contract.
getState() returns a deep copy.
- One setInterval at params.tickMs (500). stop() and reset() always clear it;
calling start/stop/reset repeatedly in any order must never create two
timers. Runs end automatically at their duration and emit runEnded once.
- Model each half-open connection as a plain object { id, source, createdAt,
expiresAt, completesAt } in an array. Per tick, in this order: expire timed-
out entries (count timedOut), generate arrivals (legit at legitRate; attack
at the level's rate, each attack SYN marked incomplete with the level's
probability using the seeded mulberry32 PRNG), apply firewall then rate limit
(outcome 'filtered'), place each surviving SYN in a free slot or, if full,
issue a SYN cookie when enabled (outcome 'cookie'; legit cookie connections
count as completed, incomplete ones cost nothing but CPU) or reject it
(outcome 'rejected'), complete legit entries whose completesAt has passed and
free their slots, then recompute metrics with the contract formulas, push to
history, derive serverState from the thresholds, and emit events.
- Emit at most 12 packet events per tick (sample proportionally between legit
and attack and include rejected/filtered/cookie outcomes) so the animation
never floods.
- Emit log events only on meaningful changes: run started/ended, queue
crossing 50%, 90%, 100%, first legit rejection, server state changes,
mitigation effects (e.g. "SYN cookies answered 120 SYNs without using queue
slots"). Never emit a fixed script of messages.
- Keep all tunable constants in one DEFAULTS object at the top with comments
explaining each cause-and-effect relationship.
- setMitigation is ignored (with a console.warn) while running.

Calibrate the constants so the Target outcomes in CONTRACT §5.4 hold. Prove
it: create tests/engine.test.js (plain Node, no libraries) that loads
engine.js in a vm with a fake window, runs each scenario synchronously by
calling an internal step function (expose it as engine._stepForTests), and
asserts the target ranges: baseline, low, medium, high, medium + each single
mitigation, medium + all mitigations, plus determinism (same settings twice →
identical summaries) and that stop() halts state changes. Run the test, print
a table of results per scenario, and iterate on constants until all pass. Do
not touch any file other than js/engine.js and tests/engine.test.js.
```

MK2 · Workflow, quiz, reset, completion (2:30–6:00)

```
Read CONTRACT.md and docs/brief.pdf (sections 4, 7, 10, 11 especially).
Implement js/workflow.js and js/quiz.js. Other files are owned by teammates:
use their public APIs (Lab.Visuals, Lab.Dashboard, Lab.Log, Lab.Content,
Lab.Icons) and never edit them. If a teammate's module is still a stub, the
page must still work without errors.

workflow.js (boots on DOMContentLoaded):
- Create one engine; call Lab.Visuals.init(engine),
Lab.Dashboard.init(engine); render Lab.Content into the guidelines list,
objective, theory, step titles/instructions, mitigation descriptions (title +
text + limitation) and component explanations.
- "Before You Begin": #btn-start disabled until #ack-checkbox is checked;
#btn-view-theory and header #btn-theory open the theory section at any time
(scroll + focus its heading). Starting hides the guidelines screen and
initialises Step 1.
- Step state machine for the 8 steps in the brief, each li's data-status =
locked | ready | in-progress | completed, with the step panel (#step-title,
#step-instructions, #step-actions) showing the current step. Gating: Step 2
baseline must finish before the flood controls unlock; Step 3 starts the
attack at the chosen level; Step 4 completes once the server has reached
Degraded or Unavailable OR the attack run ends; Step 5 completes when the
student presses Stop (#btn-stop-sim or header #btn-stop) or the run ends;
Step 6 requires at least one mitigation enabled and a run via #btn-run-
mitigated at the same level; Step 7 shows Lab.Dashboard.renderComparison({
baseline, attack, mitigated }); Step 8 runs Lab.Quiz.
- Disable controls that make no sense in the current state (e.g. no starting
a run while one is running, no mitigation changes while running). Keyboard
accessible; move focus to the new step heading when a step changes.
- Stop: halt immediately, keep results visible, log it.
- Reset (#btn-reset): if any results exist, open #modal-confirm first. On
confirm: engine.reset(), Lab.Visuals.reset(), Lab.Dashboard.reset(),
Lab.Log.clear(), Lab.Quiz.reset(), uncheck mitigations, clear stored runs,
return to Step 1 (guidelines stay acknowledged). #btn-restart / #btn-restart-
lab do the same but return to the Before You Begin screen.
- Log every workflow event with Lab.Log.add (lab initialised, baseline
generated, flood started at level X, stopped by student, mitigation enabled:
names, comparison ready, quiz finished, experiment completed).
- Completion: open #modal-complete ONLY when baseline, attack, mitigated run,
comparison and quiz are all done. Fill baseline/attack/mitigated availability
(avgAvailability from summaries), quiz score and status. #btn-review closes
the modal and scrolls to the comparison.

quiz.js: Lab.Quiz.init(container, questions, onFinished) renders one question
```

```
at a time with 4 radio-style option buttons, a Submit button, immediate
feedback (.is-correct / .is-wrong, show the explanation, and for a wrong
answer also show which option was right and why), a Next button, a progress
label "Question 2 of 6", final score in #quiz-score, and calls onFinished({
score, total }). Fully keyboard accessible (fieldset/legend, focus moves to
feedback).

After coding, open index.html?mock=1 in a headless browser if available (or
reason through it) and walk the full journey in tests/checklist.md order,
fixing anything broken. Make sure repeated start/stop/reset clicks never
throw and never leave a timer running.
```

MK3 · Integration and final build (6:00–7:00)

```
All teammates have pushed. Merge branches pranav, yash, sanika, meet into
main (their files don't overlap; if git reports a conflict, stop and tell me
which file). Then:
1. Read CONTRACT.md. Search every JS and CSS file for ids, classes, events
and API names that are NOT in the contract or are used but never defined;
list mismatches and fix them in the smallest way, preferring to fix the
caller. Report every cross-file change you made.
2. Open index.html WITHOUT ?mock=1 and walk the complete student journey in
tests/checklist.md. Check the browser console is clean. Fix integration bugs.
3. Verify: numbers on screen come from engine state; Stop halts updates
instantly; Reset returns everything to the initial state; the completion
popup cannot appear early; no network requests are made (no
fetch/XHR/WebSocket/external URLs anywhere, including fonts).
4. Run node tests/engine.test.js and node build.js. Open dist/index.html and
repeat a quick journey to confirm the 3-file build works from file://.
5. Zip dist/ as syn-flood-virtual-lab.zip and summarise what was fixed.
```

7. Pranav's prompts (Claude Code)

Pull `main` after Manvith's **MK0** push, switch to branch `pranav`, and run Claude Code in the repo. Work with `index.html?mock=1` open in Chrome until the real engine lands.

PK1 · Topology, packets and handshake demo (0:30–2:30)

```
Read CONTRACT.md and docs/brief.pdf (sections 5 and 6 especially). Implement
js/visuals.js: Lab.Visuals as specified in CONTRACT §5.8. Only edit
js/visuals.js. Use only the engine's public API and events (§5.2) and test
against the mock engine by opening index.html?mock=1. No libraries; must work
from file://.

Topology (inside svg#topology-svg, built with document.createElementNS,
responsive via viewBox):
- Five nodes with classes from §5.6: Legitimate Client (.node--client),
Simulated Traffic Generator (.node--generator), Target Server (.node--
server), TCP Connection Queue (.node--queue, attached to the server) and
Network Status Indicator (.node--status). Icons come from Lab.Icons (inline
SVG strings); if an icon is empty fall back to a labelled rounded rect. Every
node has a visible text label.
- Links (.link) from client and generator to server.
- Queue visual: a grid of state.queue.capacity small slots (.queue-grid /
.queue-slot--free/--legit/--attack) updated each tick from state.queue
counts, plus a "pending / capacity" label.
- Status node shows the current serverState text and colour class.
- Clicking or pressing Enter/Space on a node (tabindex="0", role="button",
aria-pressed) toggles .is-selected and shows Lab.Content.components[key]
title + text in #component-info.

Packets: on each 'packet' event animate a small circle along the right path
using requestAnimationFrame: SYN travels source → server; then by outcome:
'completed' → SYN-ACK back to the client and ACK to the server (.packet--
synack, .packet--ack); 'queued' attack → SYN-ACK goes out toward the
generator and fades out (no ACK ever returns); 'rejected' → packet turns
.packet--rejected and bounces/fades at the server; 'filtered' → fades at the
link midpoint with .packet--filtered; 'cookie' → .packet--cookie styling. Cap
concurrent animated packets at 60 (drop the oldest), animate only while the
tab is visible, and use one shared rAF loop. Respect prefers-reduced-motion
by showing instant state changes instead of moving dots.

Handshake demo: Lab.Visuals.playHandshakeDemo() draws a small client ↔ server
diagram inside #handshake-demo and animates SYN → SYN-ACK → ACK with captions
taken from Lab.Content.theory.handshake, then shows "Connection established".
It can be replayed (wire nothing to buttons yourself; workflow.js will call
it, but auto-play once when #handshake-demo first scrolls into view using
IntersectionObserver).
```

```
reset(): cancel all animations, clear packets, empty the queue grid, clear
selection and #component-info. Add brief comments explaining the animation
logic. Finally list anything you needed from the contract that was missing.
```

PK2 · Dashboard, chart, log, comparison (2:30–6:00)

```
Read CONTRACT.md and docs/brief.pdf (sections 7 steps 4 and 7, 8 and 9). Implement js/dashboard.js: Lab.Dashboard and Lab.Log exactly as in CONTRACT §5.8. Only edit js/dashboard.js. Test with index.html?mock=1, then with the real engine once it's merged.

Dashboard (subscribe to engine 'tick', 'serverStateChange', 'log'):
- Update the 8 [data-metric] cards each tick: Incoming SYN/s, Pending (half-open) with "x / capacity", Completed legitimate connections, Rejected or timed-out, Queue utilisation, Server resource usage (CPU), Avg response time (ms), Service availability (%). Each card fills .metric__value and, where it's a percentage, a .progress__bar width. Format numbers with toLocaleString. Make sure screen readers don't get spammed: only #notifications is aria-live.
- #server-state: text + .server-badge--healthy/--pressure/--degraded/--unavailable, plus the one-line explanation from Lab.Content.serverStates.
- On serverStateChange to Degraded or Unavailable show a toast via notify() (and a calmer one when it recovers to Healthy).
- Chart in canvas#chart-history, drawn with the 2D canvas API (no libraries): x = simulated seconds, two lines for availability % and queue utilisation % on a 0–100 axis, with axis labels, a small legend, colours read from CSS variables via getComputedStyle (--color-healthy / --color-attack etc.), crisp on high-DPI screens, redrawn on resize. Label it "Simulated data".
- Throttle DOM writes with requestAnimationFrame so a tick causes at most one paint.

Lab.Log.add(message, level): prepend an <li class="log-entry log-entry--{level}"> to #log-list with two timestamps: wall clock (HH:MM:SS) and simulated time (t = 12 s) when a run is active. Engine 'log' events are added automatically. Lab.Log.clear() empties it.

Lab.Dashboard.renderComparison({ baseline, attack, mitigated }): fill table#compare-table with rows Pending connections (peak), Legitimate connection success (%), Avg response time (ms), Peak server resource usage (%), Service availability (avg %), Final server state; columns Baseline / Under attack / With mitigation ("—" for missing runs), with a ▲/▼ change indicator in the mitigated column relative to the attack column. Then write #compare-explanation automatically from the actual numbers in 3–4 plain sentences (e.g. "During the medium flood availability fell from 100% to 22% because pending connections reached 100/100. With SYN cookies and shorter timeouts it recovered to 94%."), naming the mitigations used from summary.mitigation, and ending with a reminder that no defence fully prevents
```

```
real-world DoS.

reset(): zero all cards, badge back to Healthy, clear chart and comparison.
Comment the important logic.
```

PK3 · Bug-fix pass (6:00–7:00)

```
Read CONTRACT.md and tests/checklist.md. I'll paste the bug list from our QA
testers below. For each bug: reproduce it by reading the code path, fix it in
the owning file (prefer js/visuals.js and js/dashboard.js; if the fix belongs
in another teammate's file, make the smallest possible change and list it
clearly so the owner knows). Then check: no console errors through the whole
journey, no timers or rAF loops left running after Stop or Reset, layout
readable at 360 px, 768 px and 1366 px widths, all interactive elements
reachable by Tab with a visible focus ring. Summarise every change.

BUGS:
<paste bug list here>
```

8. Yash's and Sanika's prompts (design)

Yash and Sanika design in Figma first, then turn the designs into CSS with the Claude.ai chat. In every chat, **attach `CONTRACT.md`, `index.html` and `docs/brief.pdf`** from the repo (download them from GitHub) so Claude knows the exact ids and class names. If Pranav or Manvith has spare Claude Code time, the same prompts work there too.

Shared visual direction (agree on this at 0:00):

- Look: a clean, professional university lab tool, not a hacker-movie screen. Light theme by default, with a dark theme through `prefers-color-scheme`.
- Colour meaning, used everywhere: legitimate traffic = teal/green, attack traffic = red-orange, pending/half-open = amber, rejected = grey, completed = blue. Server states: Healthy green, Under Pressure amber, Degraded orange, Unavailable red.
- Fonts: system font stack only (`system-ui`, `Segoe UI`, `Roboto`…) and a monospace stack for numbers. **No Google Fonts or any external URL**, because the brief forbids external network traffic.
- Accessibility: text contrast at least 4.5:1, never colour alone (always a label or icon too), visible focus rings, works at 360 px width.

Yash: layout and base styles

YM1 · Figma mockup (0:30–2:00). Design three frames: desktop 1440 px, tablet 768 px, mobile 375 px. Cover the header with Stop/Reset/Restart, the Before You Begin screen, the theory section, the stepper with its four statuses, the step panel, and the two modals. Leave blank boxes for the topology and dashboard (Sanika designs those). Share the Figma link in the group by 2:00 and export a PNG of the desktop frame into the repo as `docs/mockup-layout.png`.

YM2 · base.css (2:00–5:00), Claude.ai prompt. Push the `:root` token block first, by 2:30, because Sanika's `lab.css` uses those tokens.

```
I'm the designer on a 5-person student project: a browser-only "SYN Flood
Virtual Lab" (HTML/CSS/vanilla JS). I've attached CONTRACT.md (our interface
contract), index.html (the page skeleton) and the brief PDF. I've also
attached my Figma mockup screenshot.

Write the complete css/base.css. It is concatenated with a teammate's
lab.css, so ONLY style what CONTRACT §5.6 assigns to me (Yash) plus the
global page.

1. :root tokens: define EVERY token listed in CONTRACT §5.6 with the colour
meanings below, a spacing scale, radii, shadow, system font stacks (no
external fonts or URLs at all). Add a dark theme by redefining the same
tokens inside @media (prefers-color-scheme: dark).
   Colour meanings: legit = teal/green, attack = red-orange, pending = amber,
rejected = grey, completed = blue; healthy green, pressure amber, degraded
orange, unavailable red. All text must meet WCAG AA contrast on its
background in both themes.
2. Reset/base: box-sizing, body, headings scale, links, code, a .visually-
hidden utility, :focus-visible ring using --color-focus, and @media (prefers-
reduced-motion: reduce) that disables transitions/animations globally.
3. Layout: .container (max-width ~1200px), .section spacing, the sticky
header #lab-header with title on the left and Theory/Stop/Reset/Restart
buttons on the right (Stop visually prominent, danger style). Main lab area:
on desktop a 2-column grid (left: stepper + step panel; right: topology,
controls, dashboard), collapsing to one column under 900px. Mitigation,
comparison, quiz and log stack below.
4. Components I own: .card, .btn with --primary/--danger/--ghost and
:disabled states, form controls (.field, select, .checkbox, .toggle for the
mitigation switches built on the existing checkboxes), the .stepper/.step
with the four data-status looks (locked = muted with lock icon via CSS, ready
= outlined, in-progress = primary with a subtle pulse, completed = green with
a check), .guidelines (numbered list + acknowledgement box), .theory, .modal
for <dialog> (centred, backdrop, max-width 560px, scrollable on mobile),
.toast, .sim-badge.
5. Responsive at 375 / 768 / 1440; no horizontal scrolling at 360px; touch
targets at least 44px.

Use only ids/classes from CONTRACT/index.html; do not rename anything.
Comment each section. Output the whole file in one code block.
```

YM3 · Responsive and accessibility review (5:00–6:00), Claude.ai prompt (attach the merged `index.html`, `base.css`, `lab.css` and screenshots at 375 px and 1440 px):

```
Review the attached page as an accessibility and responsive-design auditor.
Check: colour contrast of every text/background pair in both light and dark
themes, focus visibility, tab order, that no information is shown by colour
alone, touch target sizes, text overflow and horizontal scrolling at 360px,
the dialogs on mobile, and reduced-motion support. Give me a prioritised list
of problems and, for each one I own (base.css), the exact CSS fix. For
problems in lab.css or the JS, write a one-line bug report I can send to the
owner.
```

Sanika: simulation components, icons and QA

SL1 · Component mockups and icons (0:30–2:00). In Figma, design: the five topology nodes and links, packet dots for each outcome, the 100-slot queue grid (free / legit / attack), a metric card with progress bar, the four server-state badges, a log entry for each level, the comparison table, and a quiz question with correct/wrong feedback. Share by 2:00. Then generate the icons with this Claude.ai prompt:

```
I'm designing a browser-only SYN Flood virtual lab. Attached is CONTRACT.md.
Write js/icons.js exactly in the CONTRACT §5.1 wrapper format, defining
Lab.Icons with these keys: client (laptop/user), generator (traffic
generator: a box emitting arrows), server (rack server), queue (stacked
slots/inbox), status (signal/pulse), shield (defence). Each value is an
inline SVG string: viewBox="0 0 24 24", fill="none", stroke="currentColor",
stroke-width="1.75", stroke-linecap/linejoin round, aria-hidden="true",
simple line style consistent across all six, no external references. Output
the whole file in one code block.
```

SL2 · lab.css (2:00–6:00), Claude.ai prompt:

```
I'm the second designer on a student "SYN Flood Virtual Lab"
(HTML/CSS/vanilla JS). Attached: CONTRACT.md, index.html, my Figma component
screenshots, and my teammate's base.css (it defines all :root tokens; use
ONLY those var(--...) tokens, never hard-coded colours, so light and dark
themes both work).

Write the complete css/lab.css styling ONLY the classes CONTRACT §5.6 assigns
to me (Sanika):
- Topology: .topology container (aspect-ratio, responsive SVG), .node and its
five variants (icon + label, hover, .is-selected ring, keyboard focus), .link
lines, .packet dots and every variant (legit, attack, synack, ack, rejected,
filtered, cookie), using the colour meanings legit=teal/green, attack=red-
orange, pending=amber, rejected=grey, completed=blue. Variants must also
differ by shape or stroke (dashed, outlined) so colour isn't the only signal.
- .queue-grid (10×10 grid, wraps on mobile) and .queue-slot--free/--legit/--
attack with a smooth colour transition.
- Dashboard: .metric cards in a responsive grid (4 columns desktop, 2 tablet,
1 under 420px), .metric__label, .metric__value (monospace, tabular numbers),
.progress and .progress__bar (width transition), .server-badge and its 4
state variants (each with a small icon/dot and text), .chart container for
the canvas.
- .log-entry and its 4 levels (left border colour + level word), scrollable
#log-list max height.
- .compare-table (zebra rows, sticky header, horizontal scroll inside its own
box on mobile, ▲/▼ indicators coloured green/red).
- Quiz: .quiz-question, .quiz-option (large tappable buttons,
hover/focus/selected), .is-correct, .is-wrong with icons, .quiz-feedback.
- .component-info panel and .mitigation-item (switch row with title,
explanation and a muted 'Limitation' line).
Respect prefers-reduced-motion. No external URLs. Comment each block. Output
the whole file in one code block.
```

SL3 · QA testing (6:00–7:00). Run the checklist in Section 10 on Chrome desktop, a phone, and one other browser (Firefox or Edge). Post each bug in the group using the bug-report format and tag the file owner.

9. Meet's prompts (content, slides, video)

Meet owns everything the student reads, plus the demo video and slides. All of it works in the free Claude.ai chat; no Claude Code needed. Attach `CONTRACT.md` and `docs/brief.pdf` to each chat.

MM1 · content.js (0:30–3:30), Claude.ai prompt:

```
I'm writing all the text for a student "SYN Flood Virtual Lab" (a browser-only simulation; no real attack traffic). Attached: our CONTRACT.md and the professor's brief. Write js/content.js that defines Lab.Content EXACTLY in the shape of CONTRACT §5.7, wrapped in the §5.1 code wrapper, as valid JavaScript.

Audience: second- or third-year engineering students with mixed networking knowledge. Tone: clear, friendly, precise, short sentences, no jargon without a one-line explanation. Use "simulated" consistently; never claim a real attack happens.

Content required:
- title and objective (from the brief: "Demonstrate the effect of a SYN Flood attack on network availability"), and a disclaimer saying all numbers are simulated educational estimates, not real benchmarks.
- guidelines: the 9 "Before You Begin" instructions from the brief, lightly polished.
- theory: tcp, the three handshake messages (SYN, SYN-ACK, ACK), synFlood (include half-open connections, the backlog/queue, spoofed source addresses, and why the server waits ~30 s), availability, and an analogy (prank pizza orders that are never collected filling the counter so real customers are turned away). 60–120 words each.
- components: client, generator ("Simulated Traffic Generator"), server, queue ("TCP Connection Queue"), status ("Network Status Indicator"): 1–2 sentences each on its role.
- steps: the 8 steps from the brief with a title and 2–4 sentence student instruction each, telling them exactly what to click and what to observe.
- serverStates: one line each for Healthy, Under Pressure, Degraded, Unavailable.
- mitigations: synCookies, rateLimit, shortTimeout, firewall, each with title, a simple explanation (2–3 sentences), and an honest limitation (e.g. SYN cookies still cost CPU and bandwidth; rate limiting can block legitimate bursts; short timeouts can drop slow genuine clients; firewalls can't identify well-spoofed traffic).
- quiz: exactly 6 multiple-choice questions, 4 options each, answer = index of the correct option, explanation of 2–3 sentences. Cover: the handshake order, why half-open connections consume resources, the effect of a SYN flood on availability, the purpose of SYN cookies, the difference between attack and legitimate traffic, and one question on why no single defence fully prevents DoS. Make wrong options plausible.
- completion: title "Experiment Completed Successfully!" and the message from
```

```
brief section 10.

Escape quotes correctly so the file runs. Output the whole file in one code
block. Then list any facts you're unsure about so I can check them.
```

Check the facts against last week's research report before pushing; Manvith can do a 5-minute review at Sync 1.

MM2 · README and slides (4:00–6:00). README Claude.ai prompt:

```
Write README.md for our GitHub repo "syn-flood-lab": a browser-only SYN Flood
Virtual Lab (HTML/CSS/vanilla JS, no backend, no real network traffic).
Sections: one-paragraph overview; screenshot placeholder; features (guided 8-
step experiment, live topology with animated packets, real-time dashboard,
mitigation controls, before/after comparison, quiz, completion popup); how to
run (open dist/index.html in any modern browser, works offline); how the
simulation works (queue of 100 half-open slots, timeouts, the four defences;
all values are simulated estimates); project structure and team with roles:
Manvith Karkera (engine, workflow, integration), Pranav Koradiya
(visualisation and dashboard), Yash Masaye (UI design and layout), Sanika
Lobo (component design, icons, QA), Meet Mangaonkar (content, video,
presentation); ethics note that it sends no traffic and is for education
only; references (RFC 4987; Bernstein's SYN cookies page; Wang, Zhang and
Shin, INFOCOM 2002). Keep it concise.
```

Slides (8–10, in Canva or Google Slides): title + team · the problem in one picture · the TCP handshake · how our simulation models it (queue, timeouts) · lab walkthrough screenshots · results (baseline vs attack vs mitigated table from the lab) · defences and their limits · what we learned · demo video and Q&A. For speaker notes, ask Claude.ai: *"Turn the attached README and research report into speaker notes for these slides, 40–60 words each."*

MM3 · Demo video (6:00–8:00). Script prompt:

```
Write a 2.5-minute screen-recording script with voice-over for a demo of our browser-based SYN Flood Virtual Lab. Format as a table: timestamp, what is on screen, exact narration (short sentences). Flow: hook (5 s: "What happens when a server is asked to wait for thousands of people who never show up?") → Before You Begin and theory with the handshake animation → baseline traffic, healthy server → start a Medium SYN flood, queue fills, availability drops, server Degraded → Stop → enable SYN cookies + shorter timeouts, rerun → comparison table and automatic explanation → quiz and completion popup → closing line naming the team. Say clearly that all traffic is simulated inside the browser.
```

Record with OBS Studio or Xbox Game Bar (Win + G) at 1080p, add the voice-over, edit in CapCut or DaVinci Resolve with captions and a title card naming all five members, and export MP4. A 30-second cut for Instagram or LinkedIn is a bonus.

10. QA checklist and definition of done

This list is also `tests/checklist.md`. Yash and Sanika run it at 6:00 on desktop Chrome, a phone and one other browser; Manvith runs it again on `dist/index.html` before freezing.

Student journey

- [ ] Page opens from `file://` with no console errors and makes no network requests (DevTools → Network is empty)
- [ ] Before You Begin shows objective + 9 guidelines; Start is disabled until the checkbox is ticked; View Theory works
- [ ] Theory shows TCP, the handshake (animation plays and replays), SYN flood, availability and the analogy
- [ ] Stepper shows Locked / Ready / In Progress / Completed correctly at every stage
- [ ] Step 2 baseline: handshakes animate, server stays Healthy, baseline values recorded; flood controls stay locked until it finishes
- [ ] Step 3 Low / Medium / High each give a visibly different result (Under Pressure / Degraded / Unavailable)
- [ ] Queue grid fills with attack slots; pending, CPU and response time rise; availability falls; legit packets get rejected once the queue is full
- [ ] A notification appears when the server becomes Degraded or Unavailable
- [ ] Stop halts everything instantly and results stay on screen

- [ ] Each mitigation has an explanation and a limitation; rerunning with defences improves availability; no text claims 100% protection
- [ ] Comparison table shows baseline / attack / mitigated with a correct automatic explanation
- [ ] Quiz gives feedback per answer and explains wrong answers; score shown
- [ ] Completion popup shows the three availability values, score and status, and only appears after everything is done
- [ ] Log entries reflect what actually happened, with timestamps

Robustness and quality

- [ ] Rapidly clicking Start / Stop / Reset / changing level never throws or creates two running simulations
- [ ] Reset asks for confirmation when results exist, then returns everything to the start state
- [ ] Restart Lab returns to Before You Begin
- [ ] Same settings twice give the same numbers (deterministic)
- [ ] Everything reachable and usable with the keyboard; focus ring visible
- [ ] Readable with no horizontal scroll at 360 px; looks right at 768 px and 1440 px
- [ ] Dark mode looks right; reduced-motion setting stops packet animation
- [ ] Every number on the dashboard is labelled "Simulated data"

Definition of done: every box above is ticked on `dist/index.html`, `node tests/engine.test.js` passes, the three-file `dist/` is zipped, and the demo video, README and slides are ready.

If time is left over (in this order):

1. A "Download results" button that prints the comparison and quiz score via `window.print()` with a print stylesheet, so students can submit it.
2. A light/dark theme toggle in the header that overrides the system setting.
3. A "Custom" attack level with sliders for SYN rate and incomplete fraction, for curious students.

Risks and fallbacks

Risk	Fallback
Real engine late	Demo and design work continue on `?mock=1`; Manvith prioritises the engine over the quiz
A teammate's file breaks the page	Every module must fail safely; Manvith temporarily reverts that file to its stub at integration
Canvas chart takes too long	Show the last 30 values as a simple bar strip with CSS instead
Claude.ai output doesn't match the contract	Paste the error and the relevant contract section back into the same chat and ask it to fix only that